

Advanced Python Programming

Coding Challenge 1

Αποστολόπουλος Ιωάννης

ΜΠΣΠ2503

Main Exercise

1) Stating the problem

We are given an $m \times n$ dimension matrix of integers A . The goal is to find the longest increasing path in this matrix, starting from any point in it, where by “increasing path” we mean an increasing sequence of adjacent elements of the matrix.

Constraints:

1. The path must be strictly increasing, i.e. for a path $\dots \rightarrow A_{ij} \rightarrow A_{kl} \rightarrow \dots$, we must have $A_{ij} < A_{kl}$.
2. The adjacent elements of A_{ij} are $\{A_{i+1,j}, A_{i-1,j}, A_{i,j+1}, A_{i,j-1}\}$, (as long as $0 \leq i < m$ and $0 \leq j < n$), i.e. we cannot move diagonally or perform wrap-around's on the matrix.

2) Constructing a strategy

A first, naïve approach to the problem would be to seek for the brute-force solution and see where that leads us to. That would be to count the length of every possible increasing path in the matrix and return the longest one. However, this would be far from an optimal solution, since it would require an enormous amount of runtime. Specifically, starting at a cell, there are up to four possible moves, and for every such move, the search expands to a new position. In the worst case, this expansion can continue for as many steps as there are cells in the grid. That results in an exponential runtime, and when applying it to every cell, we get an asymptotic runtime of

$$O(mn \cdot 4^{mn}).$$

Thus, although valid, the brute-force approach is nowhere near our goal of optimizing the problem.

The first important observation is that the problem naturally involves recursive dependencies.

Terminology – notation: from now on, “solution of cell (i, j) ” refers to the length of the longest increasing path starting from position (i, j) . We will denote this solution as “LIP(i, j)”.

With this clarified, the solution of cell (i, j) depends on the solutions of its adjacent cells: if the element $A_{i,j}$ is (without loss of generality – WLoG) smaller than the element to its right, then every increasing path that starts at (i, j) and proceeds to the right consists exactly of the paths that originate from $A_{i,j+1}$, plus the step $(i, j) \rightarrow (i, j + 1)$. The same logic applies to all other directions in which we can move from cell (i, j) .

Therefore, the solution for cell (i, j) is:

$$\text{LIP}(i, j) = 1 + \max(\text{LIP}(x) : x \in \{(i, j + 1), (i, j - 1), (i + 1, j), (i - 1, j)\}) \quad (1)$$

while respecting all constraints of the problem.

3) Constructing a recursive solution

Here, we will outline a recursive approach in pseudocode (using Python indexing). We will be somewhat lenient with condition checking, as the goal here is to develop the main idea.

Suppose we want to return the length of the longest increasing path starting from cell (i, j) – that is $LIP(i, j)$. To construct the algorithm, we need to clarify two points:

- What is the recursive step
- What are the base cases

The first point has already been addressed in equation (1).

Regarding the second point, we have to determine under what circumstances the recursion cannot go any deeper, and therefore must return a result. This translates to the question “When is it impossible to make any further move?”, and the answer follows directly from the problem’s constraints: the recursion ends when we reach a cell whose value is larger than those of all its adjacent cells, and/or when the cell lies at the borders of the matrix (so there are no neighbors to check).

Question: What about infinite loops? How can we be sure that the recursion will not re-enter the path that we are currently constructing? In the situation described above, we have not explicitly prevented revisiting a cell that’s already included in the path under construction.

Answer: Actually, we have. The path is strictly increasing, so its last element is guaranteed to be greater than those preceding it, and thus none of them can be revisited.

So, let’s make a first attempt to implement a first version of the recursive algorithm. The input will be a cell of the matrix, and the output will be the length of the longest increasing path spawning from this cell. (Note that this code is a part of the larger program that will take as input the matrix, so when we call $A_{i,j}$, it can indeed be accessed).

Let the element (i, j) be a . Initially, the length of $LIP(i, j)$ is 1, since the chain spawning from this cell includes this cell itself.

Our first action is to attempt to move one cell in a direction. If this move cannot be made (either because it leads outside of the matrix or the destination cell holds a value not greater than a), moving towards this direction cannot contribute anything to our path, and in this preliminary work we will just ignore the move completely. If the move is valid, then our chain is guaranteed to grow, and we now have the task to compute the $LIP((i, j) + \text{move})$.

Pseudocode:

```
LIP((i, j))

max_len = 1
moves = {(0,1), (0,-1), (1,0), (-1,0)}
for move in moves:
    if ((i, j) + move) out of matrix or  $A_{((i,j)+\text{move})} \leq a$ :
        pass
    else:
        max_len = max(max_len, LIP((i, j) + move) + 1)
return max_len
```

This represents an early form of recursion that we might apply to solve the problem.

Performance:

In terms of runtime, this algorithm is no better than the brute force approach we presented in section 2. For every cell passed into LIP, four checks are performed, and each valid check will spawn a new recursive call to an adjacent cell. In the absolute worst case, where every cell on our matrix is reached by a branch of the recursive tree, we get a runtime of 4^{mn} . Applying the function to every cell yields the same asymptotic runtime as the brute force approach:

$$mn \cdot 4^{mn}.$$

Thus, additional work is needed to optimize the solution.

4) Optimizing the algorithm

The key observation that allows us to reduce the complexity of the algorithm is the presence of the property of “overlapping subproblems”, which indicates to the deployment of dynamic programming.

The main idea is the following. Suppose we finish processing cell (i, j) . During this computation, we do not only determine $\text{LIP}(i, j)$, but also the LIP values of some of its adjacent cells. WLoG, we may compute $\text{LIP}(i, j + 1)$. But, this exact computation is going to be performed repeatedly – for instance, when later computing $\text{LIP}(i, j + 2)$.

These repeated evaluations of $\text{L}(i, j)$ are what drive the running time to exponential levels. Yet, they can be considered unnecessary after $\text{LIP}(i, j)$ is computed once. Dynamic programming solves this issue.

Each time we compute LIP for a cell (i, j) , we will store this value in a container. Then, when this value is needed again during the process of LIP, instead of spawning a new recursion tree, we will simply retrieve it from this container. A hash map is ideal for this container, since checking whether an entry exists in it can be done in constant time.

This simple step will reduce the runtime of our program to linear (with respect to the matrix size). An intuitive way to see this is the following:

All operations performed via LIP at a certain level of the recursion, excluding the recursive calls themselves, take only constant time.

With the implementation of dynamic programming, each cell's LIP value is computed once. Any further need for this value can be satisfied in constant time via a lookup in our hash map anytime.

Therefore, we perform a constant amount of work for each of the $m \times n$ cells.

Overall, the optimized algorithm has an asymptotic runtime of $O(mn)$, a significant improvement over the early, original recursive algorithm.

5) Constructing the final algorithm

The implemented algorithm is included in the `.py` file. Below are some questions, obstacles, and difficulties that arose during its construction, along with how each was addressed.

1) What's the best type of container for storing the computed LIP values?

Answer: A dictionary is ideal. Before calling $LIP(i, j)$, we want to check whether its value has already been computed, so we avoid any redundant work. After each execution of LIP, we store the result in the container.

The data structure most suitable for this task is a dictionary, since inserting a new entry, checking whether a key exists, and accessing an existing value all take constant time.

2) How should we check the constraints?

Answer: The main question is whether we should:

Option 1: check all constraints before calling $LIP((i, j) + \text{move})$, or

Option 2: call $LIP((i, j) + \text{move})$ first, and then check the constraints.

If we choose Option 2, then after calling LIP, we would need to check boundary conditions using an "if" statement:

$$0 \leq i \leq m - 1 \quad \text{and} \quad 0 \leq j \leq n - 1$$

But checking the second condition (the value constraint) would require keeping track of the value from which we arrived at element (i, j) , so we can compare it with the current cell. This means we would have to introduce an additional argument, such as `preceding_value`,

and checking if it is smaller than the current value. If it's not, we could return 0, as this direction does not contribute to our path.

With Option 1, we simply verify whether a move in a given direction satisfies all the conditions before making the recursive call. If the move violates any rule, we skip it and continue with the remaining directions.

In our implementation, we chose Option 1.

3) What are our actions when a move is valid?

Answer: We proceed to compute LIP at the new position. WLoG, assume that the move to the right is valid.

Our first step is to take advantage of dynamic programming: we check whether $LIP(i, j + 1)$ has already been computed. If it has, we simply use the stored value. If not, we compute $LIP(i, j + 1)$, which deepens the recursion tree.

Once we obtain $LIP(i, j + 1)$, we determine whether this value contributes to the maximum-length path starting from (i, j) . We compare:

- the current greater path length from (i, j) , and
- the value $LIP(i, j + 1) + 1$ (the “+1” accounts for the move from (i, j) to $(i, j + 1)$)

and update the maximum accordingly.

4) At which point should we add the solution $LIP(i, j)$ to our hashmap?

Answer: Although this is trivial when realizing the logic, it posed a challenge while building the code. Specifically, should we:

- add this value to our dictionary at the end of LIP, right before returning it, or
- update the dictionary every time we update the current length of the LIP starting from (i, j) , i.e., right after executing:

```
res = max(res, LIP(i + move[0], j + move[1]) + 1)
```

The logical choice is to insert the new entry at the end, when the final value of $LIP(i, j)$ is known. While the algorithm would still work if we updated the dictionary multiple times, this would cause unnecessary operations. The asymptotic runtime would not change, since updating a dictionary entry requires constant time, but these operations would be redundant.

6) Final solution

The final algorithm can be found in the .py file, along with analytical comments regarding the purpose of each code block. Also, some example matrices as well as some edge cases are included.

Additional Questions

0) Statistical Analysis of Longest Increasing Paths

Before addressing the additional questions, it is useful to present some basic statistical results obtained from a series of tests. The function developed for the main problem was executed on a large collection of matrices of varying sizes, yielding the following outcomes:

0).1 Summary Table

Matrix Type	Avg LIP	Max LIP	Avg Duration	% Outliers
Small Matrices ($2 \leq m, n \leq 10$)	5.318	9	–	6.0%
Medium Matrices ($10 \leq m, n \leq 100$)	8.997	13	0.0075	0.1%
Large Matrices ($100 \leq m, n \leq 1000$)	11.4	13	1.2069	0.0%
Med. Matrices with High Element Variance	10.579	15	–	1.1%

Table 1: Statistics of the Longest Increasing Path (LIP) for random matrices of different sizes. Outliers are defined as LIP lengths falling outside the expected range.

0).2 Observations

- Small matrices typically have an average LIP of around 5.3, with a small percentage of outliers (6%).
- Medium matrices show an average LIP near 9, with very few outliers (0.1%).
- Large matrices exhibit longer LIPs (average 11.4) and essentially no outliers.
- Medium matrices with high variance in element values have slightly longer LIPs (average 10.58), with minimal outliers (1.1%).
- As expected, the computational duration increases with matrix size.
- On average, the length of the LIP in an $m \times n$ matrix appears to grow roughly as $\ln(mn)$.

These results will prove useful in the construction of subsequent algorithms, particularly in situations where implementing a fully robust algorithm that always generates the desired matrix is challenging. In such cases, a "trial and error" approach may be practical, as the statistical patterns observed above provide a reliable guideline.

Discussion on Preserving Structural Properties in Random Matrices

All of the following questions involve constructing random matrices while ensuring that certain properties are preserved. As discussed in class, these matrices cannot be completely random, since in that case there would be no guarantee that the desired properties would hold. This raises two important questions: “At which stage of the matrix construction should randomness be introduced?” and “What is the minimum degree of randomness we can sacrifice while still preserving the desired properties?”

The first question can be addressed using several construction strategies. One approach is to first select $m \times n$ elements and then arrange them in the matrix in such a way that the desired property holds. Another strategy is to choose the elements for each position sequentially, while simultaneously ensuring that the property is maintained. A third approach is to first fill the matrix with the elements that directly enforce the property and then populate the remaining cells, taking care not to violate the property in the process. In the solutions presented below, we implement a selection of these strategies.

It could be argued that all of the strategies mentioned above are essentially equivalent, but this is a rather mathematical point that does not concern us here.

Regarding the second question, we do not claim that our solutions achieve the theoretical optimum; nevertheless, in the examples below, the resulting matrices seem genuinely random at first sight, while still preserving the required properties.

Note that in the algorithms presented, we assume that the user provides valid input arguments; for instance, they will never request an LIP of length ≥ 10 in a 3×3 matrix.

1) Random Matrix with LIP Following a Given Pattern

To address this case, we decided not to design separate algorithms for specific patterns, nor attempt to build a generalized algorithm that could “guess” the path shape from abstract user arguments such as “snake” or “stairs” (if that is even possible). Instead, the user is required to explicitly provide the path shape as a sequence of moves that the LIP must follow.

The initial construction of the path is straightforward. We start at a chosen cell in the matrix, and following the user-provided pattern, we fill each successive cell with a value larger than the previous one. This results in an increasing path within an otherwise empty matrix.

At this stage, one might think to simply fill the remaining cells with random values. However, this approach introduces two significant problems:

1. **Preventing longer unintended paths:** How can we ensure that, while filling the remaining cells, no other increasing path emerges that is longer than the one we constructed?
2. **Maintaining the integrity of the constructed path:** How can we ensure that our path does not connect to external elements? For instance, the second-to-last cell of the path (with value a) could be adjacent to a cell with a value greater than a , which could spawn a new chain and extend the LIP beyond the intended pattern. Similarly, an external path could enter our path and disrupt its structure.

To address the first problem, there are two strategies:

1. **For long patterns relative to the matrix size** (see section 0 above): We can fill the remaining cells completely at random, potentially multiple times, and (using the function designed for the main problem) return the first matrix whose LIP equals the length of the pattern plus one (to account for the starting cell). If another path of equal length accidentally appears, it is acceptable, as it does not violate the exercise requirements.
2. **For short patterns:** We restrict the range of values used to fill the remaining cells so that the maximum possible length of any unintended path cannot exceed the length of the constructed chain.

The second problem was dealt with using a “barrier” approach. We create a barrier surrounding the path, making sure no exits or entrances appear. After many failed attempts, which are not worth mentioning, we arrived at a solution that may not be visually appealing, but certainly works:

- For every cell in the path *except the last*, we surround its neighbors (excluding cells already in the path) with a fixed value A . This value is larger than the maximum value of the path. This ensures that no external path can enter the constructed path (since all moves must be increasing), and that elements within the path can extend outside by at most one step.

- The last cell of the path is treated differently: instead of surrounding it with a larger barrier value, we fill its neighbors with values equal to its own. This prevents any moves into or out of the path, and thus any extensions of it. One may argue, "why not barrier all cells with this method?". This is exactly one of the failed attempts we tried before arriving at this final barriering method.

The procedure can be summarized as follows:

1. Construct the path starting from a chosen cell.
2. Barrier the path according to the strategy described above.
3. Fill the remaining cells:
 - Use completely random values if the pattern length is large relative to the matrix (making it unlikely for another longer path to appear). If needed, create more than one matrix and return the first one that satisfies the constraints.
 - Otherwise, use a controlled range if the pattern is short, ensuring no longer path can accidentally form.

The complete algorithm is implemented in the `.py` file named `Challenge_1_Additional_1`, and several example patterns are included, such as snake, inwards and outwards spirals, among others.

2) Random Matrix With LIP in a Given Range

To address this case, we want to generate a matrix of given dimensions such that the longest increasing path (LIP) falls within a user-specified range [lower, upper].

The procedure can be divided into two cases based on the desired LIP relative to the matrix size:

1. **Generating naturally occurring LIPs:** If the lower bound of the desired LIP is small compared to the size of the matrix (see section 0 above), it is highly likely that a random matrix will already contain a path of the required length. In this scenario, the algorithm repeatedly generates fully random matrices until the LIP computed by the function designed for the main problem falls within the desired range. This approach is simple, efficient, and works without any additional construction.
2. **Constructing the path manually:** If the lower bound of the desired LIP is large, it becomes unlikely that a purely random matrix will satisfy the constraint. In this case, the algorithm explicitly constructs a path to guarantee that the minimum length is achieved. The construction starts from the top-left cell of the matrix and follows a cyclic, clockwise movement pattern. Each cell along this path is assigned a value strictly larger than the previous one, ensuring that the path is increasing. After several attempts, we found that this approach helps ensure the path does not get “stuck” before reaching its minimum required length — imagine a snake eating its own tail, as in the popular game. While the resulting matrix may not appear entirely random when scanned, this method reliably guarantees that the desired property is preserved.

Once the path is constructed, the remaining cells are filled with random values. The algorithm ensures that the length of the LIP remains within the specified range by repeatedly generating values for the remaining cells until the computed LIP satisfies the lower and upper bounds.

The complete procedure can be summarized as follows:

1. Determine whether the desired LIP is small enough to rely on random generation or requires manual construction.
2. If necessary, construct the path to guarantee the minimum LIP.
3. Fill in the remaining cells with random values, checking that the LIP stays within the desired range. Repeat the filling process if needed.

The complete algorithm is implemented in the `.py` file named `Challenge_1_Additional_2`, and examples are included, covering small, medium, and large matrices with timing measurements to illustrate performance.

3) Random Matrix With predetermined length of LIP

In this case, the goal is to generate an $m \times n$ matrix such that the longest increasing path (LIP) has a predetermined length, specified by the user as **target**.

The procedure can be divided into three main cases, depending on the value of **target** relative to the matrix size:

1. **Target = 1:** If the desired LIP is equal to 1, the solution is trivial. Every cell in the matrix is assigned the same value, ensuring that no strictly increasing path longer than 1 can exist. This guarantees that the computed LIP matches the target without further computation.
2. **Target is small compared to the matrix size** (see section 0): For small values of **target**, it is possible that a random matrix already contains a path of the required length. In this scenario, the algorithm repeatedly generates random matrices, with each cell's value chosen from a range of size **target** - by limiting the range this way, we ensure that the LIP will not extend for more cells than that. After each generation, the LIP is computed using the function of the main problem, and the process repeats until the LIP matches the target. This probabilistic approach is efficient for small LIP lengths and avoids unnecessary manual construction.
3. **Target requires explicit path construction:** For larger values of **target**, when random generation is unlikely to yield the desired LIP, the algorithm explicitly constructs a path of length **target**. The construction starts at the top-left cell and proceeds in a cyclic, clockwise movement pattern. Each cell along this path is assigned a value strictly larger than the previous one, ensuring a strictly increasing path. The coordinates of the path are stored to prevent revisiting the same cell, and after the path is fully constructed, the same barrier mechanism that was applied to problem (1), is implemented to the adjacent to the chain cells to prevent accidental extensions of the path.

Once the path is constructed, the remaining cells in the matrix are filled with random values that do not exceed the safe range determined by the starting value of the path and the target length. This ensures that no new increasing path longer than the intended **target** can form. The overall procedure can be summarized as follows:

1. Determine whether the target LIP is equal to 1, small enough for random generation, or requires manual construction.
2. If necessary, construct a path of strictly increasing values to guarantee the LIP length.
3. Apply a barrier mechanism around the path to prevent accidental extensions.
4. Fill in the remaining cells with values that preserve the maximum LIP length, ensuring that the computed LIP matches the target.

Note: The value chosen as the threshold for deciding whether to use Case 2 or Case 3 is **10**. This choice is consistent with the conclusions of section 0, but it also follows from extensive empirical testing. We observed that Case 3 may fail not only when the target is too small compared to the matrix size, but occasionally (rarely, but still) also when the matrix itself is small. After thorough experimentation, we found that a threshold of 10:

- guarantees that the constructed LIP remains exactly equal to the target,
- avoids the rare edge cases in which Case 3 fails,
- and keeps the overall computation fast for all matrix sizes.

The complete algorithm is implemented in the `.py` file named `Challenge_1_Additional_3`, with examples illustrating small, medium, and large matrices. The implementation ensures correctness for all target lengths and prevents accidental increases in the LIP due to random filler values.

4) Random Matrix with LIP falling in a specific Sub-Region

Here the goal is to construct an $m \times n$ matrix whose Longest Increasing Path is forced to lie entirely within a user-specified sub-region of the matrix. The sub-region must be given by the user, and is defined by four bounds:

$$\text{region} = [\text{lower}, \text{upper}, \text{left}, \text{right}],$$

which describe the rows and columns between which the increasing path is permitted to "move".

Thus, the sub-region turns out to be rectangular. If we want to restrict the path to lie within a non-rectangular area, user has to specify the bounds of the largest rectangle that fits entirely within this area.

The algorithm proceeds as follows:

1. A starting point is chosen at random from inside the sub-region.
2. The algorithm constructs the LIP by performing a random walk inside the region, always selecting among the legal moves

$$\{(1, 0), (-1, 0), (0, 1), (0, -1)\}$$

that stay inside the region and do not revisit any previously visited cell.

3. Each new cell in the chain is assigned a value strictly larger than the previous one, ensuring that the constructed path is strictly increasing.
4. Once the chain cannot be extended further, we barrier it using `barrier_path()`, the same function applied to problems (1) and (3). As always, this is done in order to avoid the path extending to some unwanted direction, in this case, outside of the region.
5. All cells that do not belong to the chain or the barrier are filled with values from a range of length the same as the length of the LIP, ensuring that no increasing path with length larger than our constructed one can occur.

The resulting matrix therefore satisfies the desired property, the LIP is fully contained within the specific region

Note: In the examples following the development of the algorithm that generates the desired matrix, we purposefully left out the edge case where the sub-region has size 1×1 . This case has not been omitted because it was difficult to deal with (although, indeed it was), but because such an input is, in essence, invalid; If the LIP is required to lie entirely inside a 1×1 sub-region of the matrix, then the longest increasing path of the matrix must have length 1. This can occur only when all entries of

the matrix are identical. But in that situation, the LIP is trivially present *everywhere* in the matrix, and cannot meaningfully be said to be "contained" in a specific sub-region. Nevertheless, one could choose to “accept” this case by relaxing the interpretation of the requirements, allowing multiple LIP’s throughout the matrix, as long as at least one of them lies inside the specified sub-region. Then we could simply return a $m \times n$ matrix filled with the same value in every cell. We have not included this edge case in the algorithm.

Note 2: In our construction, the increasing path continues to grow until it either "eats its own tail" or it cannot extend further without breaking the boundaries. So, we expect (and the runs confirm it) that if the sub-region specified by the user is large, then the LIP will also be long. So, even though the exercise doesn’t directly impose any constraint on the length of the LIP, the lengths of the LIP’s produced by this algorithm will not always follow the distribution presented in Section 0.

The complete algorithm is implemented in the `.py` file named `Challenge_1_Additional_4`, accompanied by a suite of tests on small, medium, and large matrices / sub-regions.

6) Constructing a Matrix with a Specified Number of LIP's

In this case, the goal is to generate an $m \times n$ matrix such that it contains a user-specified number of Longest Increasing Paths, **denoted by** p . Unlike previous problems, the difficulty here lies in controlling the number of LIPs precisely while maintaining the structural integrity of each path.

Initial Attempt: Robust Algorithm

Initially, we tried to construct a deterministic algorithm. The idea was to split the matrix into a number of rectangles (possibly greater than the number of LIP's the user has specified) so that each LIP is contained entirely within a rectangle, while at the same time preventing cross-block paths, since that could create additional, possibly longer LIP's, changing their count. We could then call a "mix" of the functions constructed in questions (3) and (4), to create paths of a certain length inside of each of these subregions. Another idea was to limit each path inside a single row or column.

However, several problems arise when trying to implement these approaches, the main one of each approach being:

- Introducing barriers to prevent cross-block paths required reducing the size of the blocks. Some blocks became too small to contain the required LIPs, making this approach infeasible for large p .
- Containing LIPs in single rows or columns would inevitably limit the amount of LIP's to $\max(m, n)$, not even accounting for barrier cells.

Thus, although ideal, the deterministic approach proved impractical (for me) to implement for general matrix sizes and user requests.

"Las Vegas" Algorithm Approach

Given the difficulties above, we implemented a **Las Vegas-style randomized algorithm**:

1. Generate random matrices repeatedly.
2. For each generated matrix, compute all LIPs using a modified version of function of the main problem. This version of the function not only returns the length of the longest increasing path, but also **the number of LIP's** in the matrix.
3. Accept the first matrix that contains at least p LIPs. If no matrix satisfies the request after many attempts, the algorithm indicates failure.

This approach is practical because:

- It guarantees correctness: any returned matrix contains at least the requested number of LIPs.
- It is flexible: works for small to large matrices and moderate p .
- It is empirically backed: statistical analysis shows that the probability of generating a matrix with a given number of LIPs is high under reasonable conditions (see tables below for more).

Our experiments demonstrate that:

- There exists an upper bound on the number of LIPs a user can ask for, depending on the matrix size. Requests exceeding this bound are impossible to satisfy.
- The probability of generating a matrix with exactly p LIPs decreases as p approaches the upper bound determined by m and n . This upper bound was set by us, after multiple tests and simulations.
- Empirical trials guide the choice of parameters for random generation, improving efficiency and success rates.

Summary

I want to note that this method does not actually *solve* the given problem. This case was the harder one to implement, and though I managed to built some versions of an algorithm that in fact works well with most of the inputs of p I tried (large ones as well), they all failed in "extreme" cases (specifically, when p is not small enough with regard to the matrix size -for example, for $p = 7$ and $mn = 36$ - or when $p \rightarrow m \times n$). The Las Vegas approach provides a practical solution. While randomized, it leverages statistical observations to reliably generate matrices with a specified number of LIPs. This method ensures:

- Correct LIP count for returned matrices,
- Reproducibility of results within known empirical bounds,
- Feasible performance for reasonable matrix sizes and user requests.

The elephant in the room is the inability of this method to handle cases where p is large.

The complete implementation can be found in the `.py` file named `Challenge_1_Additional_6`, which includes a block of code for testing the function.

Summary of Las Vegas Algorithm Results

Results for $p = 8$

Category	Range of Values	Found Matrix with at least p LIP's	Time (s)
small matrix	small-range	True	0.03
small matrix	medium-range	True	0.019
small matrix	large-range	False	0.045
medium matrix	small-range	True	0.101
medium matrix	medium-range	True	0.643
medium matrix	large-range	True	0.439
large matrix	small-range	True	0.108
large matrix	medium-range	True	1.278
large matrix	large-range	True	0.183

Results for $p = 15$

Category	Range of Values	Found Matrix with at least p LIP's	Time (s)
small matrix	small-range	True	0.231
small matrix	medium-range	False	0.751
small matrix	large-range	False	0.831
medium matrix	small-range	True	0.341
medium matrix	medium-range	True	2.766
medium matrix	large-range	True	1.605
large matrix	small-range	True	0.931
large matrix	medium-range	True	1.41
large matrix	large-range	True	20.549

Results for $p = 18$

Category	Range of Values	Found Matrix with at least p LIP's	Time (s)
small matrix	small-range	False	0.012
small matrix	medium-range	False	0.01
small matrix	large-range	False	0.009
medium matrix	small-range	True	0.055
medium matrix	medium-range	True	0.574
medium matrix	large-range	False	2.008
large matrix	small-range	True	2.246
large matrix	medium-range	False	5.68
large matrix	large-range	True	1.376

Results for $p = 20$

Category	Range of Values	Found Matrix with at least p LIP's	Time (s)
small matrix	small-range	False	0.009
small matrix	medium-range	False	0.009
small matrix	large-range	False	0.009
medium matrix	small-range	True	0.036
medium matrix	medium-range	True	1.424
medium matrix	large-range	False	3.951
large matrix	small-range	True	0.464
large matrix	medium-range	True	2.098
large matrix	large-range	False	15.838

5) Statistical Properties to achieve certain LIP

In this case, we aim to investigate the statistical properties that a matrix should exhibit for its longest increasing path to achieve a specific length. Our goal is not to construct such a matrix explicitly, but rather to determine *when*, depending on the distribution of its entries, it is more likely for a randomly generated matrix to satisfy this condition.

To this end, we conducted a series of tests to identify which statistical characteristics are more likely to produce the desired LIP length, depending on the target length in each case. We considered three different distributions—normal, uniform, and geometric—to populate the matrices, varying both the variance of the distributions and the matrix sizes.

Table 2: Statistical properties of matrices for Normal distribution (100 trials)

Range of values	Size	Avg Value Count	Avg Max Freq	Avg Min Freq	LIP Range	Most Common LIP (Occurrences)	Avg Freq in matrices with most common LIP
thin	5×5	4.92	0.4128	0.0552	3-5	4 (57)	0.2008
thin	10×10	6.02	0.3865	0.0204	3-5	4 (59)	0.1675
thin	20×20	6.92	0.3809	0.0054	4-6	5 (90)	0.1455
thin	50×50	7.82	0.3828	0.0021	5-6	5 (59)	0.1283
thin	100×150	9.04	0.3829	0.0002	5-7	6 (97)	0.1107
medium	5×5	18.66	0.1168	0.0400	3-8	5 (37)	0.0529
medium	10×10	37.67	0.0764	0.0100	5-10	7 (41)	0.0265
medium	20×20	50.68	0.0551	0.0025	7-13	8 (44)	0.0197
medium	50×50	64.24	0.0451	0.0004	8-12	10 (44)	0.0156
medium	100×150	74.25	0.0414	0.0001	10-13	10 (38)	0.0135
long	5×5	24.19	0.0640	0.0400	4-8	5 (40)	0.0417
long	10×10	87.30	0.0280	0.0100	6-11	7 (31)	0.0115
long	20×20	246.95	0.0142	0.0025	8-12	9 (38)	0.0041
long	50×50	467.98	0.0073	0.0004	9-13	10 (49)	0.0021
long	100×150	605.48	0.0052	0.0001	10-15	12 (44)	0.0017

Table 3: Statistical properties of matrices for Uniform distribution (100 trials)

Range of values	Size	Avg Value Count	Avg Max Freq	Avg Min Freq	LIP Range	Most Common LIP (Occurrences)	Avg Freq in matrices with most common LIP
thin	5×5	9.94	0.1952	0.0420	3-7	5 (44)	0.1019
thin	10×10	11.00	0.1412	0.0465	5-9	6 (51)	0.0909
thin	20×20	11.00	0.1168	0.0677	6-9	7 (57)	0.0909
thin	50×50	11.00	0.1009	0.0815	7-10	8 (73)	0.0909
thin	100×150	11.00	0.0947	0.0872	8-10	9 (71)	0.0909
medium	5×5	22.28	0.0860	0.0400	4-9	5 (39)	0.0461
medium	10×10	64.15	0.0425	0.0100	6-10	7 (37)	0.0156
medium	20×20	99.21	0.0246	0.0025	7-12	9 (38)	0.0101
medium	50×50	101.00	0.0153	0.0055	9-14	10 (43)	0.0099
medium	100×150	101.00	0.0120	0.0079	10-13	11 (47)	0.0099
long	5×5	24.71	0.0504	0.0400	4-9	6 (34)	0.0402
long	10×10	95.13	0.0213	0.0100	6-12	8 (33)	0.0105
long	20×20	330.86	0.0088	0.0025	7-12	9 (38)	0.0030
long	50×50	918.85	0.0036	0.0004	9-14	10 (35)	0.0011
long	100×150	1001.00	0.0020	0.0003	10-14	12 (44)	0.0010

Table 4: Statistical properties of matrices for Geometric distribution (100 trials)

Range of values	Size	Avg Value Count	Avg Max Freq	Avg Min Freq	LIP Range	Most Common LIP (Occurrences)	Avg Freq in matrices with most common LIP
thin	5×5	15.17	0.1632	0.0400	3-9	5 (36)	0.0693
thin	10×10	27.97	0.1198	0.0100	5-10	6 (42)	0.0361
thin	20×20	41.15	0.1067	0.0025	6-11	8 (49)	0.0242
thin	50×50	58.48	0.1019	0.0004	8-11	9 (54)	0.0170
thin	100×150	74.92	0.1000	0.0001	9-13	10 (53)	0.0135
medium	5×5	7.60	0.3424	0.0412	2-6	4 (57)	0.1308
medium	10×10	11.55	0.3050	0.0106	4-7	5 (58)	0.0885
medium	20×20	15.59	0.2966	0.0026	5-8	6 (66)	0.0640
medium	50×50	20.76	0.2999	0.0004	6-9	7 (72)	0.0481
medium	100×150	25.79	0.3002	0.0001	7-9	8 (63)	0.0390
long	5×5	4.97	0.4980	0.0548	2-5	3 (55)	0.2267
long	10×10	7.01	0.4979	0.0138	3-5	4 (70)	0.1490
long	20×20	9.04	0.5033	0.0030	4-6	5 (63)	0.1117
long	50×50	11.87	0.4985	0.0005	5-6	5 (66)	0.0842
long	100×150	14.27	0.5005	0.0001	5-7	6 (89)	0.0704

Conclusion

Based on the statistical analysis of matrices filled with the selected distributions, we can derive the following conclusions regarding the likelihood of a longest increasing path (LIP) matching a desired length:

Effect of distribution on LIP length:

- **Normal distribution:** Produces matrices with values clustered around the mean, especially for low and medium variance (corresponding to thin and medium ranges of values). Matrices with low variance (thin range) tend to have fewer unique numbers and higher frequency repetition, often leading to shorter LIPs. Increasing the variance (medium or large range) increases the number of unique values and reduces repetition, making longer LIPs more likely.
- **Uniform distribution:** Values are evenly spread across the selected range, with the frequencies of each element approximately equal, as expected. As the range increases, the number of unique values rises quickly while the frequencies of values decrease, favoring longer LIPs.
- **Geometric distribution:** Skewed toward smaller values, with occasional large numbers depending on the success probability p . Low p (more large values) produces greater variation and a higher chance of long increasing sequences. High p (more small values) leads to many repetitions and shorter LIPs.

Effect of value repetition and unique counts:

High frequency of repeated numbers tends to limit LIP length, as consecutive duplicates do not increase the path. Matrices with more unique numbers and lower maximum frequency of values tend to have longer LIPs.

Balancing randomness and constraints:

- To favor longer LIPs while maintaining randomness: Choose distributions with wider ranges or moderate skew to increase uniqueness, and limit repetition by increasing variance (or selecting low p in the geometric case).
- To favor shorter LIPs: Use distributions with small variance or high skew toward small values, which increases repetitions.